

Algorithmic Summarization Engine

Mohammad Shojaei - Sepehr Darvishi

فهرست مطالب

۱. مقدمه و تعریف مسئله
 - ۱-۱. تعریف مسئله
 - ۱-۲. اهمیت مسئله
 - ۱-۳. ورودی ها و خروجی ها
۲. تحلیل مسئله
 - ۲-۱. دسته بندی مسئله
 - ۲-۲. چالش های الگوریتمی
 - ۲-۳. نقش مدل های زبانی
۳. طراحی الگوریتم ها
 - ۳-۱. الگوریتم کلاسیک (Text Rank)
 - ۳-۲. ماژول LLM
 - ۳-۳. الگوریتم ادغام
۴. تحلیل پیچیدگی
 - ۴-۱. تحلیل زمانی
 - ۴-۲. تحلیل فضایی
- ۴-۳. تحلیل بدترین و بهترین حالت
۵. مثال های دستی
۶. دیاگرام سیستم
۷. طراحی تست ها
۸. ساختار گیت هاب

مقدمه و تعریف مسئله

تعریف مسئله: در دنیای امروز که با انفجار اطلاعات و حجم عظیمی از داده‌های متنی روبه‌رو هستیم، استخراج اطلاعات کلیدی در کمترین زمان ممکن به یک ضرورت تبدیل شده است. مسئله‌ی اصلی این پروژه، طراحی و پیاده‌سازی یک «موتور خلاصه‌ساز خودکار» است که بتواند یک متن طولانی را به عنوان ورودی دریافت کرده و نسخه‌ای کوتاه‌تر، منسجم و حاوی نکات اصلی را ارائه دهد. تفاوت بنیادین این پروژه با روش‌های سنتی، استفاده از رویکرد هیبریدی (Hybrid) است؛ یعنی ترکیب دقت ریاضیاتی الگوریتم‌های کلاسیک (مانند TextRank) با توانایی درک معنایی مدل‌های زبانی بزرگ (LLM) برای رسیدن به بالاترین کیفیت خلاصه‌سازی.

اهمیت مسئله: خلاصه‌سازی خودکار نه تنها در زمان صرفه‌جویی می‌کند، بلکه در حوزه‌هایی مانند تحلیل اخبار، مرور مقالات علمی و مدیریت محتوای شبکه‌های اجتماعی کاربرد حیاتی دارد. اهمیت ویژه این پروژه در رفع شکاف بین «آمار» و «معنا» است. الگوریتم‌های کلاسیک در تشخیص ساختار ریاضی متن قوی هستند اما از درک ظرافت‌های معنایی عاجزند، در حالی که مدل‌های زبانی (LLM) بسیار هوشمند هستند اما گاهی دچار توهم (Hallucination) می‌شوند یا کنترل دقیقی روی طول خروجی آن‌ها وجود ندارد. ترکیب این دو، سیستمی قابل اعتماد و دقیق ایجاد می‌کند.

ورودی‌ها و خروجی‌ها:

- **ورودی (Input):** یک رشته متنی (String) با طول نامشخص که شامل چندین پاراگراف است. سیستم باید قابلیت پردازش متون با ساختارهای مختلف (مانند مقالات یا گزارش‌ها) را داشته باشد.
- **خروجی (Output):** یک متن خلاصه شده که تعداد جملات آن بر اساس پارامترهای تنظیمی (مثلاً ۳ جمله برتر یا درصدی از متن اصلی) انتخاب شده است. این خروجی باید علاوه بر حفظ ترتیب منطقی، بیشترین شباهت محتوایی را با اصل متن داشته باشد.

تحلیل مسئله

دسته بندی مسئله: این مسئله در رده‌ی خلاصه‌سازی استخراجی (Extractive Summarization) قرار می‌گیرد که با استفاده از مفاهیم نظریه گراف (Graph Theory) و پردازش زبان طبیعی (NLP) حل می‌شود. از منظر الگوریتمی، ما با یک مسئله رتبه‌بندی (Ranking) روبرو هستیم که در آن هر جمله به عنوان یک گره (Node) در نظر گرفته شده و اهمیت آن بر اساس روابطش با سایر جملات (Edges) سنجیده می‌شود. در بخش دوم، مسئله به یک مدل بهینه‌سازی هیبریدی تبدیل می‌شود که در آن امتیاز نهایی هر جمله، تابعی از وزن گرافیکی و شباهت معنایی (Semantic Similarity) است.

چالش‌های الگوریتمی: در مسیر طراحی این سیستم با چندین چالش فنی روبرو هستیم:

- **چالش شباهت‌سنجی:** استفاده از روش‌های ساده مانند اشتراک کلمات ممکن است جملاتی که کلمات متفاوت اما معنای یکسانی دارند را تشخیص ندهد. به همین دلیل استفاده از Vectorization (مانند TF-IDF یا Embedding) ضروری است.
- **پیچیدگی زمانی:** در متون بسیار طولانی، ساخت ماتریس شباهت (با مرتبه $O(n^2)$) می‌تواند زمان‌بر باشد که نیاز به پیاده‌سازی بهینه (Library Mode) دارد.
- **حفظ انسجام:** خلاصه نهایی نباید مجموعه‌ای از جملات پراکنده باشد؛ لذا حفظ ترتیب اصلی جملات پس از استخراج، یک چالش پیاده‌سازی است.
- **بالانس وزن‌ها:** پیدا کردن نقطه بهینه بین خروجی الگوریتم کلاسیک و خروجی LLM (تنظیم پارامترهای Alpha و Beta) برای جلوگیری از سوگیری سیستم.

نقش مدل‌های زبانی (LLM): در این پروژه، مدل زبانی (LLM) نقش یک **ناظر هوشمند (Oracle)** را ایفا می‌کند. برخلاف سیستم‌های ساده که فقط به آمار کلمات تکیه می‌کنند، ما از LLM برای استخراج یک "خلاصه طلایی" استفاده می‌کنیم. سپس از این خلاصه برای بازنگری در امتیازات الگوریتم کلاسیک بهره می‌بریم. به عبارت دیگر، LLM به ما کمک می‌کند تا بفهمیم کدام بخش‌های متن از نظر معنایی غنی‌تر هستند، حتی اگر تکرار کلمات در آن‌ها زیاد نباشد. این نقش باعث می‌شود جملاتی که در گراف رتبه متوسطی داشتند اما از نظر معنایی توسط LLM مهم تشخیص داده شده‌اند، به صدر لیست منتقل شوند.

طراحی الگوریتم ها

در این بخش، معماری سه لایه‌ی سیستم خلاصه‌ساز (موتور کلاسیک، ماژول هوش مصنوعی و الگوریتم ادغام) به تفصیل تشریح می‌شود. هدف نهایی این طراحی، پوشش دادن ضعف‌های ذاتی روش‌های آماری با استفاده از درک معنایی LLM است.

۳-۱. بخش اول: الگوریتم کلاسیک (TextRank)

برای استخراج جملات کلیدی بر اساس ساختار آماری متن، از الگوریتم **TextRank** استفاده شده است. این الگوریتم یک روش بدون ناظر (Unsupervised) و مبتنی بر گراف است که از الگوریتم PageRank گوگل الهام گرفته شده است.

منطق عملکرد: ایده اصلی این است که اگر جمله‌ای شباهت زیادی به سایر جملات متن داشته باشد، نقش یک «گره مرکزی» (Hub) را ایفا می‌کند و احتمالاً حاوی اطلاعات مهمی است

مراحل اجرا:

1. **پیش‌پردازش و قطعه‌بندی:** متن ورودی به جملات مجزا ($S_1, S_2, \dots, S_n$) تقسیم می‌شود.
2. **بردارسازی (TF-IDF):** برخلاف روش‌های قدیمی که فقط کلمات تکراری را می‌شمردند، ما از معیار **TF-IDF** استفاده می‌کنیم. این معیار هوشمندانه عمل می‌کند: به کلماتی که در یک جمله زیاد تکرار شده‌اند (TF) امتیاز می‌دهد، اما اگر آن کلمه در تمام جملات دیگر هم باشد (مثل فعل‌های ربطی)، امتیاز آن را با ضریب IDF کاهش می‌دهد تا کلمات خاص و متمایز برجسته شوند.
3. **محاسبه ماتریس شباهت:** برای سنجش ارتباط جملات، از **شباهت کسینوسی (Cosine Similarity)** بین بردارهای TF-IDF استفاده می‌شود. این معیار به جای طول جمله، به "زاویه معنایی" و محتوای مشترک توجه دارد.
4. **اجرای PageRank:** روی گراف حاصل، الگوریتم رتبه‌بندی اجرا می‌شود تا امتیاز نهایی هر جمله (همگرایی) مشخص شود.

سودوكد الگوریتم TextRank:

```
Function Calculate_TF(word, sentence):
    count = 0
    For each w in sentence:
        If w == word:
            count = count + 1
    Return count / Length(sentence)

Function Calculate_IDF(word, all_sentences):
    doc_count = 0
    For each sentence in all_sentences:
        If word in sentence:
            doc_count = doc_count + 1

    If doc_count == 0: Return 0

    Return Log(Total_Number_Of_Sentences / doc_count)

Function Calculate_Cosine_Similarity(Vector_A, Vector_B):
    Dot_Product = 0
    Magnitude_A = 0
    Magnitude_B = 0

    For i from 0 to Length(Vector_A) - 1:
        Dot_Product = Dot_Product + (Vector_A[i] * Vector_B[i])
        Magnitude_A = Magnitude_A + (Vector_A[i] ^ 2)
        Magnitude_B = Magnitude_B + (Vector_B[i] ^ 2)

    Magnitude_A = Sqrt(Magnitude_A)
    Magnitude_B = Sqrt(Magnitude_B)

    If (Magnitude_A * Magnitude_B) == 0:
        Return 0

    Return Dot_Product / (Magnitude_A * Magnitude_B)

Algorithm TextRank_Manual_Implementation(Document):

    Sentences = Split_Into_Sentences(Document)
    Unique_Words = Extract_Unique_Words(Sentences)
    N = Length(Sentences)
```

```

Vectors = Array of size [N][Length(Unique_Words)]

For i from 0 to N - 1:
    For j from 0 to Length(Unique_Words) - 1:
        word = Unique_Words[j]
        tf = Calculate_TF(word, Sentences[i])
        idf = Calculate_IDF(word, Sentences)
        Vectors[i][j] = tf * idf

Graph = Create_Empty_Graph()
For i from 0 to N - 1:
    For j from i + 1 to N - 1:

        Weight = Calculate_Cosine_Similarity(Vectors[i], Vectors[j])

        If Weight > Threshold:
            Add_Edge(Graph, Node_i, Node_j, Weight)

Scores = Run_PageRank(Graph)
Return Top_Sentences(Scores)

Function Run_PageRank(Graph, iterations=20, d=0.85, convergence_threshold=0.0001):
    N = Number_of_Nodes(Graph)

    Scores = Array of size N, filled with 1.0

    For step from 1 to iterations:
        New_Scores = Array of size N

        For i from 0 to N - 1:
            Sum_Neighbors = 0

            For each j in Neighbors(Node_i):

                W_ji = Get_Edge_Weight(Node_j, Node_i)

                Total_Out_Weight_j = Sum_of_Weights_of_All_Edges_from(Node_j)

                Sum_Neighbors = Sum_Neighbors + (W_ji / Total_Out_Weight_j) * Scores[j]

            New_Scores[i] = (1 - d) + d * Sum_Neighbors

    If Max_Difference(Scores, New_Scores) < convergence_threshold:

```

```
Break Loop

Scores = New_Scores

Return Scores
```

ماژول مدل زبانی (LLM Oracle): در این سیستم، مدل زبانی (LLM) نقش **نویسنده** را ندارد، بلکه نقش **مشاور (Oracle)** را بازی می‌کند. الگوریتم‌های گرافی مثل TextRank در درک "مفاهیم انتزاعی" ناتوان هستند (مثلاً نمی‌دانند "پادشاه" و "سلطان" مرتبطند مگر اینکه عین کلمه تکرار شود).

وظیفه ماژول:

1. دریافت متن کامل به عنوان ورودی.
2. تولید یک "خلاصه چکیده" (Abstractive Summary) با کیفیت بالا.
3. این خلاصه مستقیماً به کاربر نمایش داده نمی‌شود، بلکه به عنوان **"استاندارد طلایی"** (Ground Truth) به مرحله بعدی ارسال می‌شود تا الگوریتم متوجه شود "چه مفاهیمی واقعاً مهم هستند".

الگوریتم ادغام: این بخش، نوآوری اصلی پروژه در مسیر سوم (Track 3) است. ما باید خروجی "آماري" TextRank را با خروجی "معنایی" LLM ترکیب کنیم.

منطق ادغام: ممکن است جمله‌ای در متن اصلی وجود داشته باشد که حاوی اطلاعات حیاتی است، اما چون کلماتش در بقیه متن تکرار نشده‌اند، TextRank آن را حذف کرده باشد (مشکل Blind Spot). برای حل این مشکل، ما تمام جملات متن اصلی را با **خلاصه تولید شده توسط LLM** مقایسه می‌کنیم. این مقایسه نه بر اساس کلمات مشترک، بلکه با استفاده از **Embedding** (فضای برداری معنایی) انجام می‌شود.

فرمول امتیازدهی نهایی:

امتیاز نهایی هر جمله ترکیبی وزن‌دار از دو معیار است:

$$Score_{Final} = (\alpha \times Score_{TextRank}) + (\beta \times Similarity_{Embedding})$$

بخش اول (آلفا) تضمین می‌کند جمله از نظر ساختاری و آماری در متن مرکزی است.

بخش دوم (بتا) تضمین می‌کند جمله از نظر معنایی به آنچه LLM مهم تشخیص داده، نزدیک است.

سودوکد الگوریتم ادغام:

```
Algorithm Hybrid_Merge_Summarizer(Original_Sentences, TextRank_Scores,
LLM_Summary_Sentences, Alpha=0.5, Beta=0.5):

    LLM_Vectors = Generate_Embeddings(LLM_Summary_Sentences)

    Hybrid_Candidates = List()

    For i from 0 to Length(Original_Sentences) - 1:

        Statistical_Score = Normalize(TextRank_Scores[i])

        Max_Semantic_Similarity = 0

        For each llm_vec in LLM_Vectors:

            Current_Sim = Cosine_Similarity(Original_Vectors[i], llm_vec)

            If Current_Sim > Max_Semantic_Similarity:
                Max_Semantic_Similarity = Current_Sim

        Final_Hybrid_Score = (Alpha * Statistical_Score) + (Beta * Max_Semantic_Similarity)

        Add (Index: i, Score: Final_Hybrid_Score, Text: Original_Sentences[i]) to Hybrid_Candidates

    Sort Hybrid_Candidates by Score Descending

    Top_N = Select_Top_Items(Hybrid_Candidates, Count=3)

    Sort Top_N by Index Ascending

    Final_Summary_Text = ""
    For each item in Top_N:
        Final_Summary_Text = Final_Summary_Text + item.Text + " "
```

تحلیل پیچیدگی

در این بخش، کارایی سیستم پیشنهادی از دو جنبه‌ی زمان (سرعت اجرا) و فضا (میزان حافظه مصرفی) بررسی می‌شود. با توجه به اینکه الگوریتم پایه بر اساس گراف‌های متراکم (Dense Graphs) بنا شده است، تحلیل دقیق مرتبه اجرایی (\$Big-O\$) برای اطمینان از مقیاس‌پذیری سیستم حیاتی است.

تحلیل زمانی: زمان اجرای کل سیستم مجموع زمان اجرای سه ماژول اصلی است:

$$T_{Total} = T_{TextRank} + T_{LLM} + T_{Merge}$$

الف) تحلیل ماژول TextRank:

این بخش پرهزینه‌ترین قسمت محاسباتی در سمت سرور است.

1. برداری‌سازی (Vectorization): برای تبدیل N جمله با دایره لغات V ، پیچیدگی برابر است با:

$$O(N \times V)$$

2. محاسبه ماتریس شباهت (Similarity Matrix): این گلوگاه اصلی الگوریتم است. از آنجا که گراف ما کامل (Fully Connected) است، باید شباهت هر جمله با تمام جملات دیگر محاسبه شود.

$$\frac{N(N-1)}{2}$$

تعداد مقایسه ها:

$$O(N^2)$$

که از این مرتبه است:

هزینه هر مقایسه (ضرب داخلی): متناسب با طول بردارها یا دایره لغات (V).

$$O(N^2 \times V)$$

پیچیدگی ساخت گراف:

3. اجرای PageRank: اگر تعداد تکرارها را ثابت فرض کنیم (مثلاً ۲۰ بار)، پیچیدگی این بخش خطی یا وابسته به تعداد یالها (E) است. در گراف کامل $E \approx N^2$ است، بنابراین پیچیدگی این بخش نیز $O(N^2)$ خواهد بود.

(ب) تحلیل ماژول ادغام (Hybrid Merge): الگوریتم ادغام طراحی شده بسیار بهینه است.

1. مقایسه: هر جمله اصلی (N) با جملات خلاصه LLM (تعداد ثابت M ، مثلاً ۵ جمله) مقایسه می‌شود.

2. پیچیدگی: $O(N \times M)$. چون M بسیار کوچک و ثابت است، رفتار این بخش خطی $O(N)$ است.

نتیجه‌گیری زمانی:

از آنجا که N^2 بر N قالب است، مرتبه اجرایی کل الگوریتم برابر است با: $O(N^2)$

تحلیل فضایی: فضای حافظه (RAM) مورد نیاز برای اجرای الگوریتم به دو ماتریس اصلی وابسته است:

1. ماتریس ویژگی‌ها (Feature Matrix): نگهداری بردارهای TF-IDF یا Embedding.

فضای مورد نیاز: $O(N \times V)$ یا $O(N \times D)$ که D بُعد بردار Embedding است.

2. ماتریس مجاورت (Adjacency Matrix): نگهداری وزن یال‌های گراف.

a. از آنجا که گراف بدون جهت و وزن‌دار است، یک ماتریس $N \times N$ از اعداد اعشاری (Float)

ذخیره می‌شود.

b. فضای مورد نیاز: $O(N^2)$.

$$O(N^2 + N \cdot V)$$

پس پیچیدگی فضایی کل برابر است با:

برای متون معمول (مثلاً ۱۰۰ تا ۵۰۰ جمله)، این فضا کاملاً توسط حافظه‌های استاندارد قابل مدیریت است، اما برای کتاب‌های بسیار حجیم نیاز به شکستن متن (Chunking) خواهد بود.

تحلیل بدترین و بهترین حالت:

رفتار الگوریتم به شدت به "طول متن" (N) و "تنوع کلمات" (V) وابسته است.

- بدترین حالت (Worst Case):

- سناریو: متنی طولانی که در آن تمام کلمات منحصر به فرد هستند (تنوع واژگان V ماکسیمم است) و هیچ جمله‌ای تکراری نیست.
- تحلیل: در این حالت، ماتریس TF-IDF بسیار خلوت (Sparse) اما بزرگ است و محاسبه شباهت کسینوسی بیشترین زمان را می‌برد. همچنین ماتریس گراف $N \times N$ کاملاً پر می‌شود.
- عملکرد: کندترین زمان اجرا با رشد مربعی.

- بهترین حالت (Best Case):

- سناریو: متنی کوتاه یا متنی که جملات آن بسیار شبیه به هم هستند.
- تحلیل: اگر تعداد جملات N کم باشد، هزینه $O(N^2)$ ناچیز است. همچنین اگر از بهینه‌سازی Threshold استفاده کنیم و یال‌های با وزن کم را حذف کنیم، گراف خلوت‌تر شده و سرعت همگرایی PageRank افزایش می‌یابد.

- تأثیر تأخیر اوراکل (Oracle Latency):

- در دنیای واقعی، با وجود اینکه پیچیدگی محاسباتی TextRank از نوع $O(N^2)$ است، اما ممکن است زمان پاسخ‌دهی API مدل زبانی (Network Latency) بیشتر از زمان محاسبات ریاضی طول بکشد. بنابراین در تحلیل سیستم واقعی، زمان LLM به عنوان یک سربار ثابت بزرگ (C_{LLM}) در نظر گرفته می‌شود.